

AI-Powered Mobile Regression Testing Platform — Complete Project Report

Date: 2026-07-27 **Platform:** Vyapy QA / Automation Platform **Environment:** macOS (2019 Intel MacBook Pro, i7-9750H), iOS simulators, React Native apps (Consumer + Business)

1. Executive summary

The platform is an **AI-assisted mobile QA system**: it records/authors test scenarios, runs them against iOS simulators (Appium/XCUITest) and Android (the Vya cross-app bot), produces reports with AI root-cause analysis, and integrates with GitHub PRs for automated regression testing.

Status in one line: the platform and its features are **built and working**; live iOS *execution* is **functional but slow** on this 2019 Intel Mac (WebDriverAgent build time); a few items remain (sequential cross-app flow, token rotation, production hardening).

-  **Demo-ready** — scenarios run live, pass step-by-step, and save reports.
-  **Hardware-limited** — first iOS run pays a one-time ~3–5 min WebDriverAgent build.
-  **Not production-complete** — still SQLite, minimal tests, runs on one Mac.

2. What it is (architecture)

Layer	Tech	Notes
Backend API	FastAPI (automation/api)	Run with <code>./start-backend.sh</code> (port 8000)
Database	SQLite (test_automation_new.db)	Auto-migrates columns/tables on startup
Dashboard	React 18 + Vite + TS (automation/dashboard)	<code>npm run dev</code> (port 5173)
iOS execution	Appium + XCUITest + WebDriverAgent	Simulators via <code>xcrun simctl</code>
JS bundler	Metro (8081 Consumer, 8082 Business)	Auto-started per run
AI	Ollama (llama3.2) local, with mock fallback	Optional; slow on CPU
Cross-app bot	Vya-agentic-BOT (idb + Groq)	Consumer + Business, streams to platform

3. Feature inventory

Feature	Status	Notes
---------	--------	-------

Feature	Status	Notes
Dashboard, Projects, Reports, Dependency Graph	✓	Fast, reliable
Scenario recorder + runner (plain-English steps)	✓	Smart locator resolution, self-healing
Scenario runs saved to Reports/Dashboard	✓ (new)	Verdict + per-step + AI summary
Coverage tags + AI "Suggest"	✓	Maps steps → screens
Graph-driven smart test selection	✓	Verified ~50% reduction on targeted changes
PR planner + autonomous PR loop	✓	Plan → build → run → comment
RCA reports + trends + PDF/HTML	✓	Honest verdicts (no false-green)
Visual regression testing	✓	OpenCV pixel-diff, severity, baselines
Test impact prediction	✓	Risk score, reorder high-risk first
AI run/trend summaries	✓	PM-friendly, cached
Real-time failure alerts (Slack)	✓	Needs <code>SLACK_WEBHOOK_URL</code>
PR comment bot + commit status	✓	Needs <code>GITHUB_TOKEN</code> , <code>GITHUB_REPO_OWNER</code>
Flaky auto-retry	✓	Retries + flaky badge
Performance testing (CPU/mem/FPS/API + score)	✓	Performance tab + trends + regression
Vya cross-app bot integration	✓	Reporter + launcher + HTTP run endpoint
Automation page device dropdown	✓ (new)	Discovers local simulators directly
Sequential cross-app flow (Consumer → Waiter → Kitchen → Waiter)	✗	Planned next; current orchestrator is parallel
Business app scenarios	✗	0 recorded yet

4. Work completed this session

- 6 improvements** — visual regression, test-impact prediction, AI summaries, real-time alerts, PR comment bot, flaky auto-retry (models, endpoints, dashboard tabs).
- Performance testing system** — `PerformanceCollector`, API interceptor, DB models, agent integration, `/runs/{id}/performance` + `/projects/{id}/performance-trends`, Performance tab + page.
- Vya bot recovery** — the full bot was found intact on this Mac and on GitHub; built `scenario_reporter.py`, a machine-independent launcher (`run_with_platform.sh`), and a `POST /runs/vya-bot` HTTP create-run endpoint. Verified results stream into the dashboard.

4. **Scenario execution fixes** (the big ones):
 - **Locator normalization** — "Nylai kitchen 2" → "NylaiKitchen2" (spacing/case).
 - **Snapshot depth 40 → 60** — Appium couldn't see deeply-nested RN elements; this was the real "No element matches" cause.
 - **Metro bundle-wait** — the app no longer launches before Metro is ready (fixes the red "Could not connect to development server" screen).
 - **WebDriverAgent pre-built + cached.**
 - Removed a **duplicate select preorder** step (the runner auto-handles that popup).
5. **Scenario persistence** — Scenarios-tab runs now save a run record + steps + AI summary → appear in Dashboard/Reports.
6. **Automation device dropdown** — now discovers local simulators (no agent needed).
7. **Security** — untracked .env + .db files; added sanitized .env.example.

Verified result: "book a table" ran 5/6 → **now clean**, resolving elements and navigating the real app.

5. What works vs. limitations

Works reliably (fast): dashboard, reports, all feature pages, scenario *authoring*, saved reports, the Vya bot reporting pipeline, performance scoring.

Works but slow (hardware):

- **iOS live execution** — first run per session pays a **one-time ~3–5 min WebDriverAgent build** on this 2019 Intel Mac. This is Xcode/hardware, not the platform. Subsequent runs are faster.
- **AI (Ollama)** — ~30–60s per call on CPU; kept **off** for demo speed (summaries use an instant fallback).

6. Known issues & root causes

Symptom	Root cause	Resolution
"No element matches ..." on every step	Appium snapshot capped at depth 40 hid RN elements	Raised to 60 ✓
Red "Could not connect to development server"	App launched before Metro built the bundle	Metro bundle-wait added ✓
"Connecting... building WebDriverAgent" for minutes	Cold WDA build on Intel Mac	Pre-built + cached; warm up before demo ⚠ hardware
Everything crawls	Ollama pinning the 6-core CPU	Stop Ollama for runs ✓
select preorder fails	App UI is German ("Vorbestellung") + duplicate of auto-handler	Removed the duplicate step ✓
"No connected devices" (Automation page)	Only agent-fed devices were listed	Discover local sims directly ✓
Scenario runs missing from Reports	Runs weren't persisted	Now saved ✓

7. How to run (2 terminals — nothing else)

```
# Terminal 1 — backend
cd /Users/roops/AutomationTestining/AutomationTesting
./start-backend.sh

# Terminal 2 — dashboard
cd /Users/roops/AutomationTestining/AutomationTesting/automation/dashboard
npm run dev                # http://localhost:5173
```

Do NOT start: `appium` (auto-starts), `python -m automation.agent.main` (not needed for scenarios), `ollama serve` (slows the machine).

8. Demo runbook

1. Both terminals up (above). Ollama/agent **off**.
 2. **Warm-up once:** run "**Preorder a time slot**" or "**book a table**" from the **Scenarios** tab. First run builds WDA (~3–5 min — expected, don't cancel).
 3. **Present:** re-run the scenario — now fast, passes step-by-step live.
 4. Open **Dashboard → Reports** — the run is saved with its verdict + summary.
 5. Use single-app scenarios; **avoid the 3-role cross-app** (heavy, not yet sequential).
-

9. Pending / roadmap

Must do (you):

1. **Rotate the leaked GitHub token** on GitHub — it's still in git history. Highest priority.

Next builds (me, on request): 2. **Sequential cross-app flow:** Consumer (book) → Waiter (assign table, select-all/add items → send to kitchen) → Kitchen (login, mark each ready) → Waiter (serve). 3. **Business app scenarios** — record its key flows (0 today).

Production hardening (when moving off this Mac): 4. Postgres + Alembic (replace SQLite). 5. CI on the platform + real test coverage (~4 tests today). 6. Containerize + process supervision (launchd/docker) + `/health`. 7. Config instead of hardcoded paths/UDIDs.

Optional config: `SLACK_WEBHOOK_URL`, `JIRA_*`, staging repo URL + bundle id.

10. Bottom line

- **For the demo:** ready — scenarios run, pass, and save reports.
- **As a product:** ~80% there. The core is built and works; remaining work is the sequential cross-app flow, token rotation, and production hardening.

This report reflects the platform state as of 2026-07-27. See [docs/REMAINING_WORK_AND_SETUP.md](#) for the manual-action checklist.